Graph Traversal - Depth First Search(DFS)

A graph consists of vertices (nodes) connected by edges (lines). Graph traversal involves visiting all the graph nodes following a specific strategy or order. During traversal, each node is typically marked as visited to avoid revisiting the same node multiple times and to prevent infinite loops in cyclic graphs.

There are two common graph traversal algorithms: Depth-First Search (DFS) and Breadth-First Search (BFS). Let's briefly discuss each of them:

Depth First Search(DFS)

Depth-First Search (DFS) is a graph traversal algorithm that explores all the nodes in a graph by systematically visiting as far as possible along each branch before backtracking. It operates on both directed and undirected graphs and can be implemented using recursion or an explicit stack data structure.

DFS starts from a selected source node (or a starting point) and explores as deeply as possible along each branch before backtracking. The algorithm visits nodes in a depth ward motion until it reaches a leaf node with no unexplored neighbors. At that point, it backtracks and explores other unexplored branches.

Here's a step-by-step explanation of the DFS algorithm:

1. Initialization:

- Choose a starting node as the source node.
- Create a data structure to keep track of visited nodes (e.g., an array or a hash set) and mark the source node as visited.

2. Visit the Current Node:

- Process the current node (e.g., print its value or perform any other operation you need to do).

3. Recursive Approach (Using Recursion):

- For each unvisited neighbor of the current node:
  - Recursively call the DFS function with the neighbor as the new current node.
  - Mark the neighbor as visited.

4. Stack-Based Approach (Using an Explicit Stack):

- Push the starting node onto the stack.
- While the stack is not empty:
  - Pop a node from the stack (current node).
  - Process the current node (e.g., print its value or perform any other operation you need to do).
  - For each unvisited neighbor of the current node:
    - Push the unvisited neighbor onto the stack.
    - Mark the neighbor as visited.

5. Backtracking:

- If there are no more unvisited neighbors for the current node, backtrack by returning from the recursive function (if recursion) or popping nodes from the stack until a node with unvisited neighbors is found (if using an explicit stack).

6. Termination:

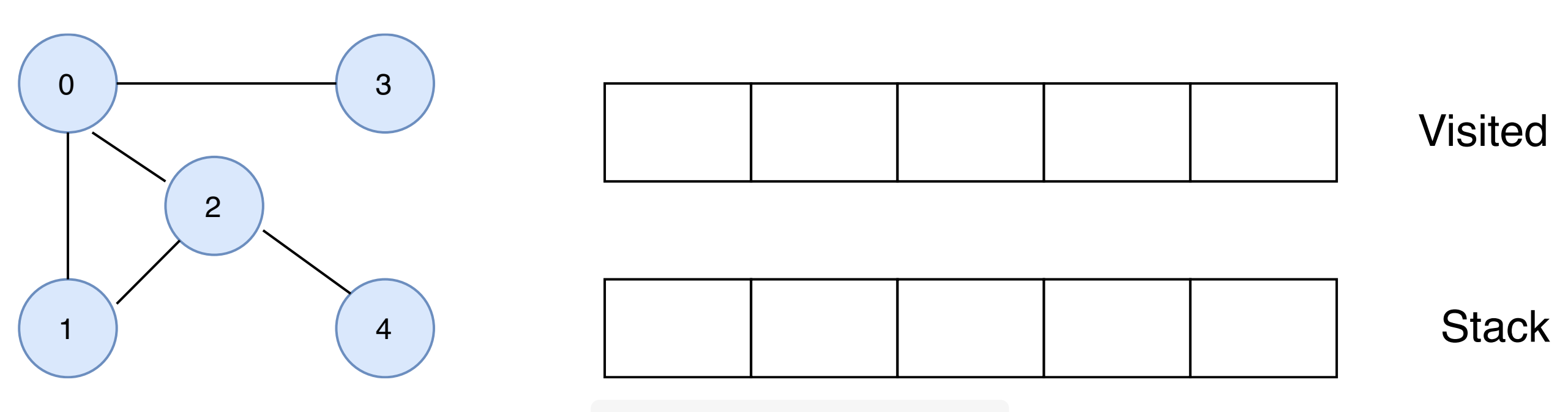
- The DFS algorithm terminates when all nodes reachable from the source node have been visited. This means that all connected components of the graph have been explored.

Step-by-step Illustration of DFS

Let's illustrate Depth-First Search (DFS) on a simple graph with its step-by-step traversal process.

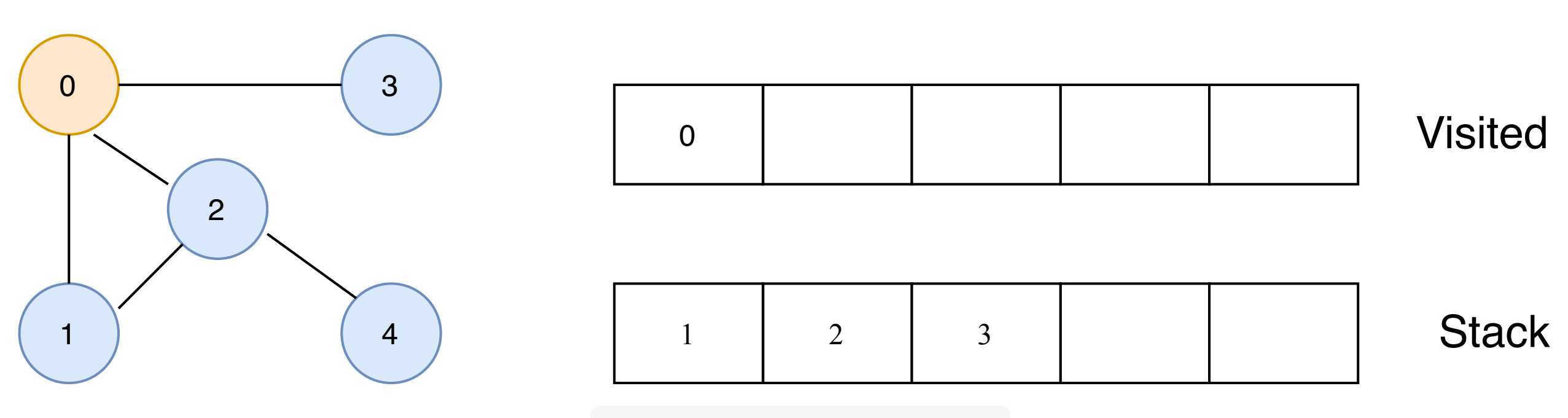
Consider the following graph:

Depth-First Search (DFS) algorithm on an undirected graph with 5 vertices



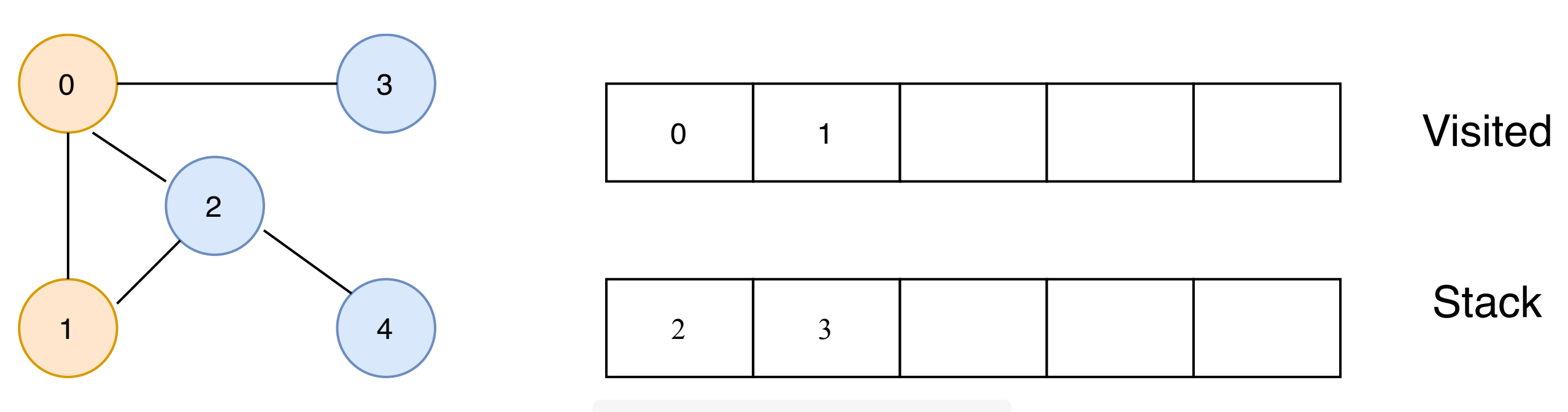
Depth-First Search (DFS)

We are starting from vertex 0, the DFS algorithm starts by putting it in the visited list and putting all its adjacent vertices in the stack.



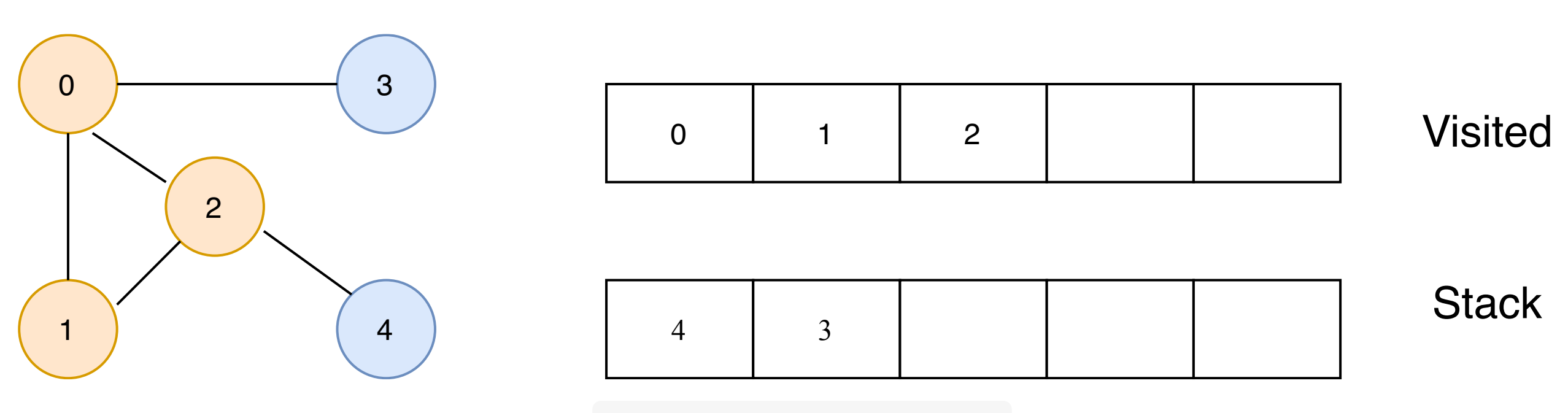
Depth-First Search (DFS)

Next, we visit the element at the top of stack i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead



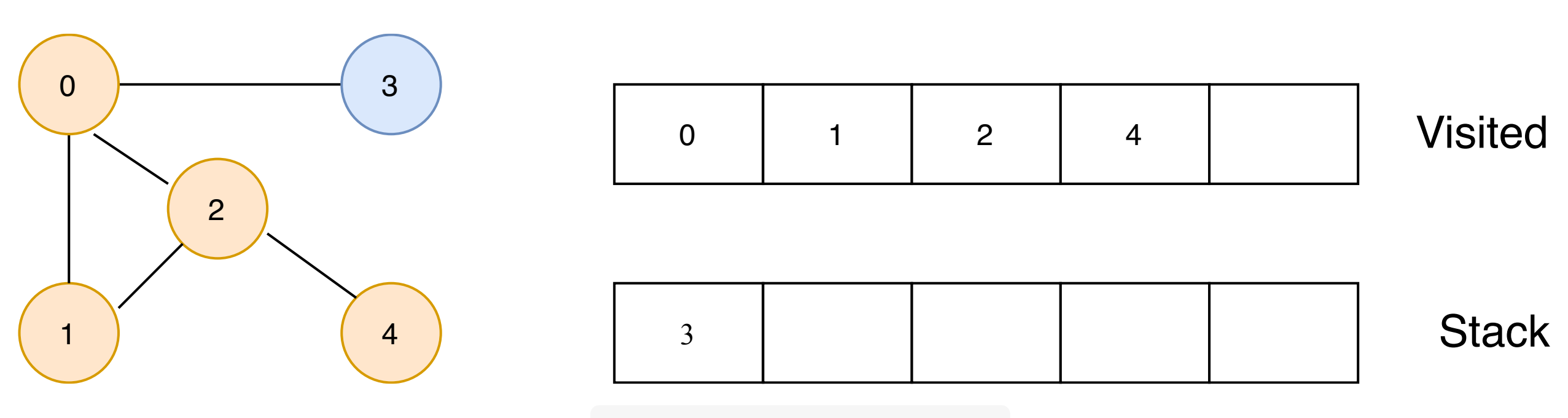
Depth-First Search (DFS)

Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the top of the stack and visit it.



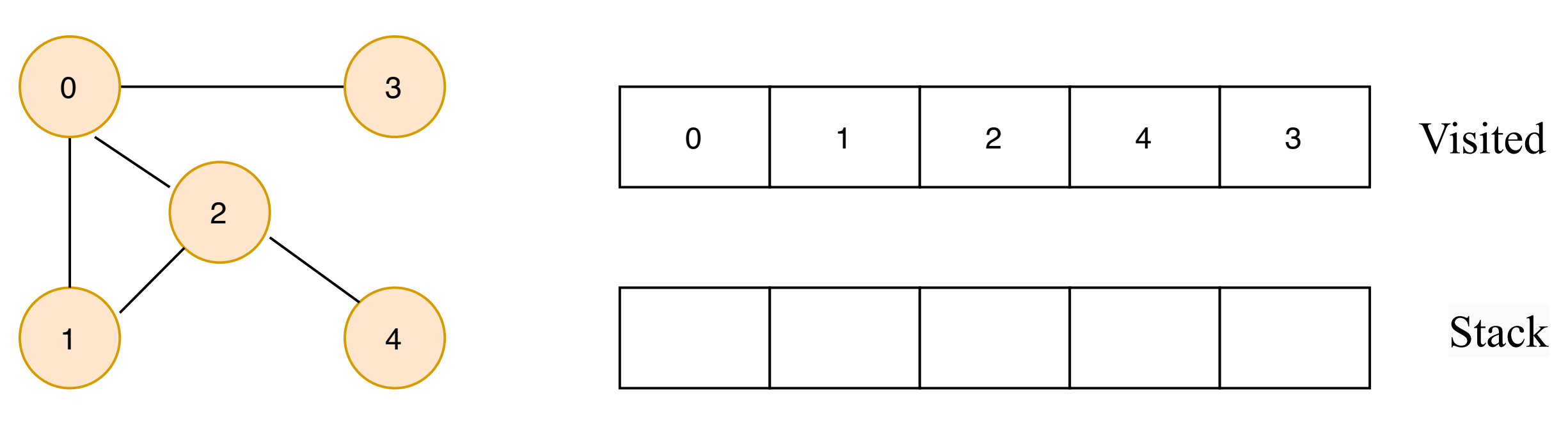
Depth-First Search (DFS)

Next, we will visit 3.



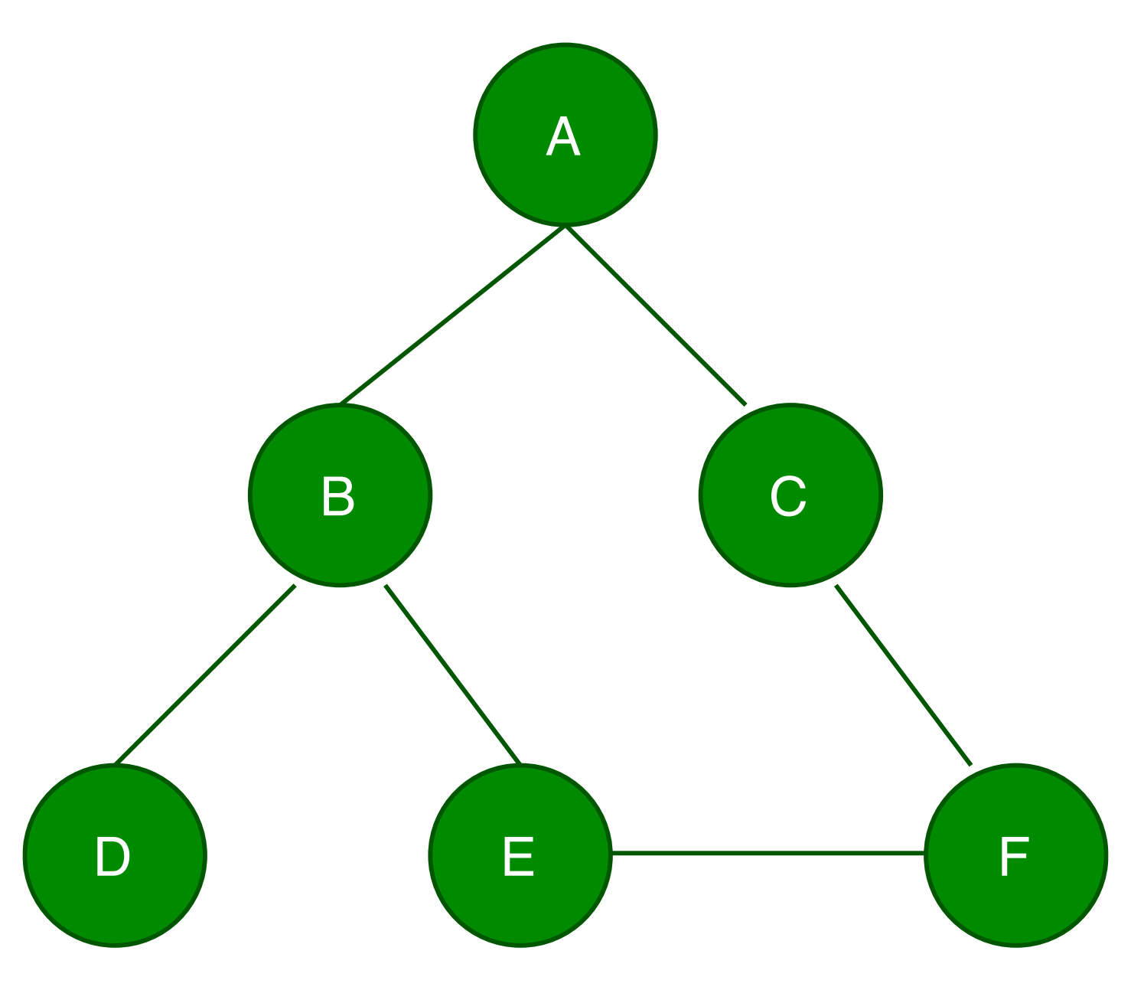
Depth-First Search (DFS)

After we visit the last element 3, it doesn't have any unvisited adjacent nodes, so we have completed the Depth First Traversal of the graph.



Depth-First Search (DFS)

Now, consider this example:



Starting from node A, let's perform DFS on this graph:

1. Start at node A (the source node).
2. Mark node A as visited and process it: A (visited).
3. Explore an unvisited neighbor of A. Let's say we choose B.
4. Mark node B as visited and process it: A -> B (visited).
5. From node B, explore an unvisited neighbor. We choose D.
6. Mark node D as visited and process it: A -> B -> D (visited).
7. Node D has no unvisited neighbors, so we backtrack to node B.
8. Node B has another unvisited neighbor, E. We explore E.
9. Mark node E as visited and process it: A -> B -> D -> E (visited).
10. From node E, explore an unvisited neighbor. We choose F.
11. Mark node F as visited and process it: A -> B -> D -> E -> F (visited).
12. Node F has no unvisited neighbors, so we backtrack to node E.
13. Node E has no more unvisited neighbors, so we backtrack to node B.
14. Node B has no more unvisited neighbors, so we backtrack to node A.
15. Node A has one more unvisited neighbor, C. We explore C.
16. Mark node C as visited and process it: A -> B -> D -> E -> F -> C (visited).
17. Node C has no unvisited neighbors, so we backtrack to node A.
18. Node A has no more unvisited neighbors, and we have visited all reachable nodes.

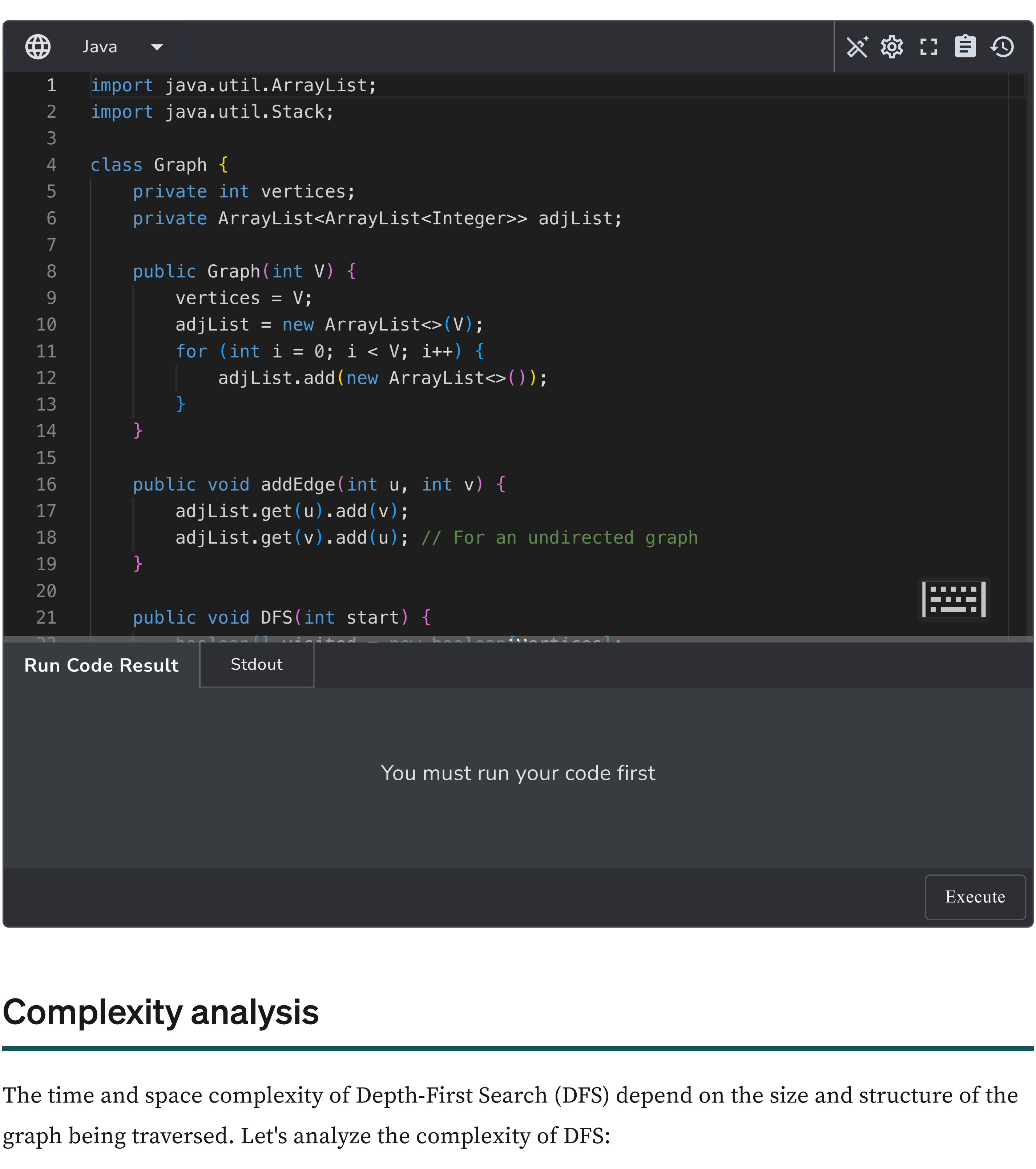
The DFS traversal order for this graph starting from node A is: A -> B -> D -> E -> F -> C.

Note that the choice of the starting node can affect the order of traversal for the graph. Also, in a disconnected graph, you would need to start DFS from each unvisited node to traverse all components.

Implementation of Depth First Search

Below is an implementation of Depth-First Search (DFS) in C++, Python, Java, and JavaScript. DFS is a graph traversal algorithm that explores as far as possible along each branch before backtracking.

In this example implementation, we assume that the graph is represented as an adjacency list.



```java
import java.util.ArrayList;
import java.util.Stack;

class Graph {
    private int vertices;
    private ArrayList<ArrayList<Integer>> adjList;

    public Graph(int V) {
        vertices = V;
        adjList = new ArrayList<>(V);
        for (int i = 0; i < V; i++) {
            adjList.add(new ArrayList<>());
        }
    }

    public void addEdge(int u, int v) {
        adjList.get(u).add(v);
        adjList.get(v).add(u); // For an undirected graph
    }

    public void DFS(int start) {
```

You must run your code first

Complexity analysis

The time and space complexity of Depth-First Search (DFS) depend on the size and structure of the graph being traversed. Let's analyze the complexity of DFS:

1. Time Complexity:

- In the worst case, DFS once visits all nodes and edges in the graph. For a graph with V vertices (nodes) and E edges, the time complexity of DFS is $O(V + E)$
- The time complexity can be further broken down as follows:
  - Visiting a node (marking it as visited and processing it) takes $O(1)$ time.
  - Exploring all neighbors of a node takes $O(d)$ time, where 'd' is the average degree of nodes in the graph. In the worst case, 'd' can be as high as $V - 1$ (complete graph).
    - So, the time complexity can be approximated as $O(V)$ for exploring all neighbors of one node.

In summary, the overall time complexity of DFS is: $O(V + E)$

2. Space Complexity:

- The space complexity of DFS is determined by the space needed to store information about the nodes during the traversal. The primary sources of space usage are the recursion stack (if using recursion) or the explicit stack data structure (if using an iterative approach).
- In the worst case, the maximum depth of the recursion stack (or the maximum number of nodes stored in the stack) is the height of the deepest branch of the graph. For a graph with a single connected component, this height can be $O(v - 1)$ (when all nodes are connected in a straight line).
- The space complexity of the recursion stack in the worst case is $O(V)$. Additionally, if an explicit stack is used, its space complexity would also be $O(V)$ in the worst case.

In summary, the overall space complexity of DFS is $O(V)$ due to the recursion stack or the explicit stack.

DFS can be used for various applications, such as finding connected components, detecting cycles in the graph, topological sorting, and solving problems like maze exploration or finding paths between nodes.

It's essential to be cautious about infinite loops when traversing graphs that may have cycles. To avoid this, the algorithm must keep track of visited nodes and avoid revisiting nodes that have already been explored.

Overall, DFS is a powerful graph traversal algorithm that can efficiently explore the entire graph and is widely used in many graph-related problems.